

Faculty of Computers and Information Systems

Information Retrieval

Practical Laboratory

Week 5: Boolean Retrieval Using Binary Vectors

Evaluate AND and NOT queries over the Week 4 incidence matrix. Trace the vectors before interpreting the matched document IDs.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Translate query terms into binary vectors.
- Apply AND and NOT operations over a fixed document universe.
- Implement query processing without external search libraries.
- Compare hand predictions with intermediate vectors and returned IDs.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

Boolean retrieval returns documents that satisfy a logical condition. AND requires all positive conditions. NOT complements a condition relative to the loaded collection. For 0/1 values, a & b performs conjunction, and 1 - bit performs complement. Python ~bit is unsuitable here because it returns negative integers rather than a 0/1 complement.

For the original collection, python is [1, 0, 0, 1, 0] and java is [0, 1, 0, 0, 1]. NOT java is [1, 0, 1, 1, 0], so python AND NOT java is [1, 0, 0, 1, 0]. The final 1 positions identify doc1 and doc4.

The supported grammar is [NOT] term (AND [NOT] term)*. Operators are case-insensitive and must be separate words. Each term contains letters only. OR, parentheses, and implicit conjunction are outside this lab. A positive unknown term gives an all-zero vector; NOT unknown gives all ones.

The original scripts split on the substring AND, which can split words such as pandas. This version splits on regex word boundaries and validates each clause. Query terms are lowercased and looked up directly; stop words have no indexed row. A stop word therefore behaves as an unknown term in this Boolean grammar.

Course material: run_query and bit-vector operations in 5.py, with token-boundary parsing and input validation added.

3. Python Concepts Used

- **re.split and fullmatch:** Split operator words and validate one optional NOT plus one term.
- **Bitwise &:** Combine corresponding 0/1 entries.
- **List copies:** Copy a row before using it as a running result.
- **enumerate:** Map result positions back to document IDs.
- **ValueError and try/except:** Report unsupported query syntax without crashing the interactive session.

4. Worked Example

Predict all three vectors in this complete example, then execute it. The term vectors match the shipped collection.

```
python_bits = [1, 0, 0, 1, 0]
java_bits = [0, 1, 0, 0, 1]
not_java = [1 - bit for bit in java_bits]
result = [python_bits[i] & not_java[i] for i in range(5)]
print(python_bits)
print(not_java)
print(result)
```

The printed complement is [1, 0, 1, 1, 0], and the result is [1, 0, 0, 1, 0]. Every vector uses the same order: doc1, doc2, doc3, doc4, doc5. NOT is always evaluated against that complete five-document universe.

5. Step-by-Step Practical Task

Step 1: Reuse the exact Week 4 docs collection and matrix builder.

Step 2: Before running, predict results for python AND data, machine AND learning, python AND NOT java, and data AND analysis.

Step 3: Run the default demonstration and compare every intermediate vector with your predictions.

Step 4: Trace `parse_query` and explain the tuple (negative, term) returned for each clause.

Step 5: Complete `and_not` in the starter. Then add a function mapping 1 positions to IDs.

Step 6: Run `python lab05_solution.py --interactive`. Test an unknown term, a negated unknown term, and invalid syntax. Type quit to exit.

Execution: open a terminal in this week folder and run `python lab05_solution.py`. On Windows, `py -3` can replace `python`. All required files are included; no internet or third-party Python packages are needed.

For repeated queries, run `python lab05_solution.py --interactive`. Type quit to exit.

6. Complete Python Program

The listing below matches `lab05_solution.py` exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab05_solution.py

```
import os
import re

STOP_WORDS = {
    "a", "an", "the", "is", "in", "for", "and", "are", "with",
    "like", "of", "to", "it", "be", "used", "uses", "that", "this",
    "was", "were", "by", "from", "on", "at", "or",
}

SAMPLE_DOCUMENTS = {
    "doc1": "Python is great for data science and machine learning",
    "doc2": "Java is used for enterprise applications and web development",
    "doc3": "Machine learning uses neural networks and decision trees",
    "doc4": "Python supports data analysis with libraries like Pandas",
    "doc5": "Java and C++ are used for game development and graphics",
}

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_documents(folder_path):
    """Load UTF-8 .txt files; use five samples if missing or empty."""
    documents = {}
    if os.path.isdir(folder_path):
        for filename in sorted(os.listdir(folder_path)):
            filepath = os.path.join(folder_path, filename)
            if filename.endswith(".txt") and os.path.isfile(filepath):
                with open(filepath, "r", encoding="utf-8") as file:
```

```

        documents[filename[:-4]] = file.read()
    if not documents:
        return SAMPLE_DOCUMENTS.copy()
    return documents

def tokenize(text):
    """Use the same ASCII-letter tokens as the supplied course files."""
    words = re.findall(r"[a-z]+", text.lower())
    return [word for word in words if word not in STOP_WORDS]

def build_matrix(documents):
    doc_names = sorted(documents)
    words_in_docs = {}
    all_words = []
    for name in doc_names:
        words_in_docs[name] = tokenize(documents[name])
        all_words.extend(words_in_docs[name])
    matrix = {}
    for term in sorted(set(all_words)):
        row = []
        for name in doc_names:
            row.append(1 if term in words_in_docs[name] else 0)
        matrix[term] = row
    return matrix, doc_names

def print_matrix(matrix, doc_names):
    print("Term".ljust(14), "|", " | ".join(doc_names))
    for term, row in matrix.items():
        values = " | ".join(str(bit).center(4) for bit in row)
        print(term.ljust(14), "|", values)

def parse_query(query):
    """Grammar: [NOT] term (AND [NOT] term)*; no OR or parentheses."""
    clauses = re.split(r"\bAND\b", query.strip(), flags=re.IGNORECASE)
    parsed = []
    for clause in clauses:
        match = re.fullmatch(
            r"(NOT\s+)?([a-z]+)", clause.strip(), re.IGNORECASE
        )
        if match is None:
            raise ValueError("Use: term AND term AND NOT term")
        term = match.group(2).lower()
        if term in {"and", "not", "or"}:
            raise ValueError("An operator cannot be used as a term")
        parsed.append((match.group(1) is not None, term))
    return parsed

def run_query(query, matrix, doc_names, trace=True):
    result = None
    for negative, term in parse_query(query):
        bits = matrix.get(term, [0] * len(doc_names))
        if trace:
            print(term, "->", bits)
        if negative:
            bits = [1 - bit for bit in bits]
            if trace:
                print("NOT", term, "->", bits)
        if result is None:
            result = bits[:]
        else:
            result = [result[i] & bits[i] for i in range(len(bits))]
    matched = [doc_names[i] for i, bit in enumerate(result) if bit == 1]
    if trace:
        print("Result vector:", result)
    return matched

def main():
    import sys
    documents = load_documents(os.path.join(BASE_DIR, "docs"))

```

```

matrix, doc_names = build_matrix(documents)
print("Document order:", doc_names)
if "--interactive" in sys.argv:
    print("Use term AND term AND NOT term; quit exits.")
    while True:
        try:
            query = input("Query > ").strip()
        except (EOFError, KeyboardInterrupt):
            break
        if query.lower() == "quit":
            break
        try:
            print("Matched docs:", run_query(query, matrix, doc_names))
        except ValueError as error:
            print(error)
    else:
        for query in [
            'python AND data',
            'machine AND learning',
            'python AND NOT java',
            'data AND analysis',
        ]:
            print("\nQuery:", query)
            print("Matched docs:", run_query(query, matrix, doc_names))

if __name__ == "__main__":
    main()

```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

Document order: ['doc1', 'doc2', 'doc3', 'doc4', 'doc5']

Query: python AND data
python -> [1, 0, 0, 1, 0]
data -> [1, 0, 0, 1, 0]
Result vector: [1, 0, 0, 1, 0]
Matched docs: ['doc1', 'doc4']

Query: machine AND learning
machine -> [1, 0, 1, 0, 0]
learning -> [1, 0, 1, 0, 0]
Result vector: [1, 0, 1, 0, 0]
Matched docs: ['doc1', 'doc3']

Query: python AND NOT java
python -> [1, 0, 0, 1, 0]
java -> [0, 1, 0, 0, 1]
NOT java -> [1, 0, 1, 1, 0]
Result vector: [1, 0, 0, 1, 0]
Matched docs: ['doc1', 'doc4']

Query: data AND analysis
data -> [1, 0, 0, 1, 0]
analysis -> [0, 0, 0, 1, 0]
Result vector: [0, 0, 0, 1, 0]
Matched docs: ['doc4']

8. Laboratory Exercises

1. Prediction: manually evaluate the four default queries and record both bit vectors and IDs.
2. Implementation: complete `and_not` without importing the solution.

3. Debugging: replace 1 - bit with ~bit in a disposable copy. Explain the negative values produced.
4. Interpretation: predict python AND unknown and python AND NOT unknown before running.
5. Parser test: run pandas and verify that it is treated as one term. Test python OR java and explain the rejection.
6. Extension: add OR as a separate optional function over two vectors, then state how you would define precedence before extending the grammar.

9. Student Tasks

- Your completed starter and manual predictions for all four default queries.
- A trace of python AND NOT java, including the original and complemented java vectors.
- Three edge-case queries with the expected and observed behavior.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. What condition does AND enforce?
2. Relative to which set does NOT operate?
3. Why is ~bit different from 1 - bit?
4. What should an unknown positive term return?
5. Why does splitting on the substring AND break pandas?

11. Challenge Exercise

Optional: Support a parenthesized expression language only after defining its grammar and precedence. Keep the simple baseline available and compare results on expressions supported by both versions.